Design and Implementation of a Scalable IoT Monitoring System Using Multi-Protocol Communication and Real-Time Visualization

Training Report Submitted in Partial Fulfillment of the Requirements
for the Degree of

Bachelor of Engineering in Information Technology

Submitted by
Govind Singh Rathore
Under the Supervision of
Assistant Prof. Arpit Khandelwal
Indian Institute of Technology, Jodhpur

Department of Computer Science and Engineering
MBM University, Jodhpur

July, 2025

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to **Assistant Prof. Arpit Khandelwal**, Indian Institute of Technology, Jodhpur, for his invaluable guidance, constant encouragement, and technical support throughout the course of this project. His insightful suggestions, constructive feedback, and mentorship played a crucial role in the successful completion of this work.

I would also like to extend my heartfelt thanks to **PhD scholars Koustav and Raj Kumar** for their continuous support, technical guidance, and valuable discussions during the internship period. Their willingness to help and share knowledge greatly contributed to my understanding and successful implementation of various aspects of this project.

I am grateful to the **Department of Electrical Engineering, Indian Institute of Technology, Jodhpur**, under which this internship was carried out, for providing an excellent research-oriented environment, necessary infrastructure, and exposure to advanced technologies in the field of Internet of Things (IoT).

I would also like to thank the faculty members and staff of the institute for their cooperation and support during the internship period.

I extend my sincere thanks to my parents, family members, and friends for their constant motivation, encouragement, and moral support, which helped me remain focused and dedicated throughout the project.

Finally, I acknowledge with gratitude all the online resources, open-source communities, and technical documentation that significantly enhanced my understanding and aided in the successful completion of this work.

Abstract

The Internet of Things (IoT) is transforming the way physical devices interact with digital infrastructure by enabling seamless, real-time data collection, communication, and intelligent automation. With the growing need for smart systems in various domains, IoT offers a foundation for building responsive, data-driven environments that adapt to changing conditions.

This project presents the design and implementation of a robust, scalable, and modular IoT-based monitoring system capable of collecting and analyzing environmental data from various sensors. The system architecture integrates a combination of microcontrollers, including the Raspberry Pi 4, NodeMCU (ESP8266), and Raspberry Pi Pico W, to facilitate distributed data acquisition, wireless communication, and edge-level processing.

Multiple sensors were employed to capture critical environmental parameters. These include the DHT11 sensor for temperature and humidity monitoring, PIR sensors for motion detection, ultrasonic sensors for distance and obstacle measurement, and infrared temperature sensors for non-contact thermal readings. These sensors collectively enable comprehensive real-time monitoring suitable for smart home, industrial, and environmental applications.

Sensor data is transmitted via optimized communication protocols—specifically HTTP and MQTT—and is processed using tools like Node-RED. For efficient storage and long-term analysis, the project utilizes InfluxDB, a time-series database. Real-time and historical visualizations are enabled using Grafana dashboards and ThingSpeak cloud analytics.

To ensure modularity, portability, and ease of deployment, the system components—such as the database, MQTT broker, and visualization tools—are containerized using Docker. This enhances reproducibility and simplifies the scaling of services in production-ready IoT solutions.

This project not only demonstrates a complete end-to-end IoT pipeline but also serves as a practical model for future smart environment systems, offering valuable insights into data-driven decision-making and system automation. .

Contents

1 Introduction **5**
1.1 Hardware & Sensor Integration . . . 6
1.2 Communication Protocols . . . 6
1.3 Data Processing, Storage, and Visualization . . . 6
1.4 Modular Deployment with Docker . . . 7

2 Sensor Interfacing with Raspberry Pi **8**
2.1 Interfacing DHT11 Sensor . . . 8
2.1.1 Working Principle of DHT11 . . . 8
2.2 Interfacing with ultrasonic Sensor . . . 9
2.2.1 Working Principle of Ultrasonic Sensor . . . 9
2.3 Interfacing with PIR (Passive Infrared) Sensor . . . 10
2.3.1 Working Principle of PIR Sensor . . . 10
2.4 Interfacing with Infrared Temperature Sensor (MLX90614) . . . 11

3 Introduction to IoT Communication Protocols **12**
3.1 Introduction to IoT Communication Protocols . . . 12
3.2 HTTP Protocol with Soil Sensor . . . 13
3.3 MQTT Protocol with Soil Sensor . . . 13
3.4 Experimental Time Analysis . . . 14

4 Advanced MQTT and QoS Level Analysis **15**
4.1 QoS 0 (At most once) . . . 15
4.2 QoS 1 (At least once) . . . 15
4.3 QoS 2 (Exactly once) . . . 15
4.4 Observations . . . 16

5 Wireless Communication Using LoRa **17**
5.1 Overview of LoRa (RA-02) module . . . 17
5.2 LED Blinking via LoRa Transmitter–Receiver Setup . . . 18

6 IoT-Based Environmental Monitoring System **20**
6.1 Hardware used: . . . 20
6.2 Data Collection and Transmission . . . 21
6.3 Node-RED for Data Flow Management . . . 21
6.4 InfluxDB for Time-Series Storage . . . 21
6.5 Grafana for Real-Time Visualization . . . 22
6.6 ThingSpeak for Cloud Monitoring . . . 22

7 Docker-Based IoT Data Processing and Visualization System **23**
7.1 Overview of Docker . . . 23
7.2 Benefits of Modular Docker-Based Deployment . . . 23
7.3 Services in Docker Containers . . . 23
7.4 Docker Container Status Output . . . 23
7.5 How Docker Works in This Project . . . 24

8 Conclusion **26**

1 Introduction

The rapid growth of the Internet of Things (IoT) is transforming how physical systems interact with the digital world by enabling intelligent, automated, and real-time data exchange. IoT plays a crucial role in modern applications such as smart homes, environmental monitoring, healthcare, industrial automation, and smart cities. By integrating sensors, microcontrollers, communication protocols, and cloud platforms, IoT systems provide actionable insights and enable data-driven decision-making. This

six-week internship focused on the design, development, and deployment of complete end-to-end IoT solutions, covering the entire pipeline from sensor data acquisition to real-time visualization and cloud-based analytics. The internship emphasized hands-on learning, allowing me to work with real hardware, implement efficient communication protocols, process live data streams, and deploy scalable IoT infrastructure using containerization technologies. Throughout the internship, I worked with multiple

microcontroller platforms including Raspberry Pi 4, NodeMCU (ESP8266), and Raspberry Pi Pico W, each selected based on application requirements such as processing capability, power efficiency, and connectivity. These devices were interfaced with various environmental and motion sensors like DHT11, PIR, Ultrasonic (HC-SR04), and MLX90614, enabling real-time monitoring of temperature, humidity, motion, distance, and object temperature. To ensure reliable and efficient data

transmission, both HTTP and MQTT communication protocols were implemented and evaluated. While HTTP provided a simple and widely supported data transfer mechanism, MQTT demonstrated superior performance for real-time and high-frequency sensor communication due to its lightweight publish–subscribe architecture. The collected sensor data was further processed, stored, and visualized

using industry-standard IoT tools such as Node-RED for data flow automation, InfluxDB for time-series data storage, and Grafana for interactive real-time dashboards. Additionally, ThingSpeak was used for rapid cloud-based visualization and MATLAB-powered data analysis. To enhance scalability, portability, and ease of

deployment, Docker containerization was used to deploy core services including InfluxDB, Grafana, and the Mosquitto MQTT broker. This modular approach ensured isolated, reproducible environments and reflected real-world IoT deployment practices. Overall, this internship provided comprehensive exposure to hardware integration,

networking protocols, data management, visualization, and DevOps practices, strengthening my practical understanding of IoT system architecture and preparing me for real-world IoT and embedded system development.

1.1 Hardware & Sensor Integration

During the internship, I gained hands-on experience with multiple microcontroller platforms including:

Raspberry Pi 4: A powerful single-board computer used for running edge applications, databases, and visualization tools.

NodeMCU (ESP8266): A Wi-Fi-enabled microcontroller suitable for lightweight IoT tasks and data transmission.

Raspberry Pi Pico W: A microcontroller with built-in Wi-Fi, ideal for low-power sensor data collection and control. These microcontrollers were interfaced with several types of sensors, each selected for specific environmental monitoring tasks:

DHT11 (Temperature & Humidity Sensor): Captures ambient temperature and relative humidity data, commonly used for indoor air quality monitoring.

PIR (Passive Infrared Sensor): Detects motion by sensing infrared radiation, typically used for occupancy detection and security systems.

Ultrasonic Sensor (HC-SR04): Measures distance using ultrasonic waves, useful in obstacle detection, smart parking, or liquid level monitoring.

Infrared Temperature Sensor (MLX90614): A non-contact sensor that measures the temperature of objects or surfaces using infrared radiation.

1.2 Communication Protocols

To transmit data collected from sensors to cloud or local servers, I implemented and compared the following protocols:

HTTP (Hypertext Transfer Protocol): A request-response protocol used for simple sensor data upload to web servers and services like ThingSpeak.

MQTT (Message Queuing Telemetry Transport): A lightweight, publish-subscribe messaging protocol ideal for real-time communication in resource-constrained environments. It proved more efficient in handling frequent sensor updates due to lower overhead and better scalability.

1.3 Data Processing, Storage, and Visualization

The sensor data was processed, stored, and visualized using a variety of modern IoT tools:

Node-RED: A flow-based development tool used to design data pipelines and trigger actions based on incoming sensor data.

InfluxDB: A high-performance time-series database used to store historical sensor data efficiently.

Grafana: An open-source platform integrated with InfluxDB for creating dynamic and interactive dashboards to visualize sensor readings in real-time.

ThingSpeak: A cloud-based IoT analytics platform used for quick data visualization, MATLAB-based analysis, and sharing data over the internet.

1.4 Modular Deployment with Docker

As IoT systems grow in complexity, managing multiple services such as databases, visualization tools, and communication brokers becomes increasingly challenging. To address issues related to dependency management, scalability, and deployment consistency, **Docker-based containerization** was adopted during the internship. Docker enables applications and their dependencies to be packaged into lightweight, portable containers, ensuring consistent behavior across different systems and environments.

During this internship, Docker was used to create a **modular and scalable IoT architecture** by containerizing essential backend services, including **InfluxDB**, **Grafana**, and the **MQTT broker (Eclipse Mosquitto).** Each service was deployed in its own isolated container, allowing independent configuration, maintenance, and scaling without affecting other components of the system.

2 Sensor Interfacing with Raspberry Pi

2.1 Interfacing DHT11 Sensor

The **DHT11** sensor is a low-cost digital sensor used to measure temperature and humidity, making it suitable for basic environmental monitoring applications in IoT systems. During the internship, the DHT11 sensor was interfaced with microcontrollers such as**NodeMCU (ESP8266)** and **Raspberry Pi Pico W** to collect real-time environmental data and transmit it to backend services for storage and visualization.

2.1.1 Working Principle of DHT11

The DHT11 sensor contains the following components:

- A thermistor for temperature measurement
- A capacitive humidity sensor for relative humidity detection

The sensor internally processes the analog signals and provides a digital output, eliminating the need for external Analog-to-Digital Converters (ADCs). Data is transmitted using a single-wire serial communication protocol, which makes interfacing simple, reliable, and efficient.

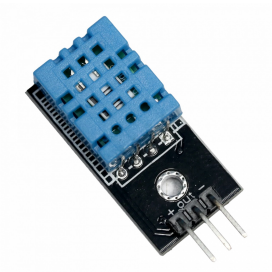


Figure 1: DHT11 Sensor Diagram

Pin Configuration

- **VCC** – Power supply (3.3V or 5V)
- **DATA** – Digital data output
- **NC** – Not connected
- **GND** – Ground

```
import Adafruit_DHT
import time

SENSOR = Adafruit_DHT.DHT11
PIN = 4

print("Reading DHT11 sensor data...")

while True:
    humidity, temperature = Adafruit_DHT.read_retry(SENSOR, PIN)

    if humidity is not None and temperature is not None:
        print(f"Temperature: {temperature:.1f}°C | Humidity: {humidity:.1f}%")
    else:
        print("Failed to retrieve data from DHT11 sensor")

    time.sleep(2)
```

Figure 2: DHT11 Sensor Interfacing Code

2.2 Interfacing with ultrasonic Sensor

The ultrasonic sensor (HC-SR04) is widely used for distance and obstacle detection in IoT and embedded systems. During the internship, this sensor was interfaced with microcontrollers such as **NodeMCU (ESP8266)**, **Raspberry Pi Pico W**, and **Raspberry Pi**, enabling real-time distance measurement for applications including smart parking systems, obstacle avoidance, and liquid level monitoring.

2.2.1 Working Principle of Ultrasonic Sensor

The HC-SR04 operates based on the *time-of-flight* principle of ultrasonic waves. The sensor emits high-frequency ultrasonic sound waves at 40 kHz and calculates the distance by measuring the time taken for the echo to return after reflecting from an object.

Figure 3: HC-SR04 Ultrasonic Sensor

Pin Configuration

Pin	Description
VCC	Power supply (5V)
TRIG	Trigger input to initiate ultrasonic pulse
ECHO	Output pin that generates a pulse corresponding to echo time
GND	Ground

```
 import RPi.GPIO as GPIO
import time

TRIG, ECHO = 23, 24
GPIO.setmode(GPIO.BCM)
GPIO.setup(TRIG, GPIO.OUT)
GPIO.setup(ECHO, GPIO.IN)

def get_distance():
    GPIO.output(TRIG, True)
    time.sleep(0.00001)
    GPIO.output(TRIG, False)

    while GPIO.input(ECHO) == 0: pass
    start = time.time()
    while GPIO.input(ECHO) == 1: pass
    end = time.time()

    return round((end - start) * 17150, 2)

try:
    while True:
        print("Distance:", get_distance(), "cm")
        time.sleep(1)
except KeyboardInterrupt:
    GPIO.cleanup()
```

Figure 4: HC-SR04 Ultrasonic Sensor Interfacing Code

2.3 Interfacing with PIR (Passive Infrared) Sensor

The Passive Infrared (PIR) sensor is commonly used for motion and human presence detection in IoT and embedded systems. During the internship, the PIR sensor was interfaced with microcontrollers such as **NodeMCU (ESP8266)**, **Raspberry Pi Pico W**, and **Raspberry Pi** to detect motion in real time for applications including security systems, smart lighting, and occupancy monitoring.

2.3.1 Working Principle of PIR Sensor

A PIR sensor detects motion by sensing changes in infrared radiation emitted by warm objects, such as humans or animals, within its field of view. The sensor consists of a pyroelectric element that generates an electrical signal when it detects a change in infrared levels. When a moving object passes through the sensor's detection zone, the change in infrared radiation triggers the sensor output.

Figure 5: PIR (HC-SR501) Sensor

Pin Configuration

Pin	Description
VCC	Power supply (5V or 3.3V)
OUT	Digital output (HIGH when motion is detected, LOW otherwise)
GND	Ground

```
#define PIR_PIN D5  // Connect PIR OUT pin to NodeMCU D5
void setup() {
Serial.begin(115200);     // Start Serial Monitor
pinMode(PIR_PIN, INPUT);   // PIR sensor as input
Serial.println("PIR Sensor Test Started...");
}
void loop() {
int pirValue = digitalRead(PIR_PIN); // Read PIR sensor
if (pirValue == HIGH) {
Serial.println("Motion Detected!");
} else {
Serial.println("No Motion");
}
delay(500); // Delay to reduce serial spam
}
```

Figure 6: PIR (HC-SR501) Sensor Interfacing Code

2.4 Interfacing with Infrared Temperature Sensor (MLX90614)

The MLX90614 is a non-contact infrared temperature sensor used to measure the temperature of objects and surfaces. During the internship, it was interfaced with microcontrollers such as NodeMCU (ESP8266), Raspberry Pi Pico W, and Raspberry Pi for real-time temperature measurement, suitable for applications like industrial monitoring, medical screening, and smart automation.

Working Principle

The sensor detects infrared radiation emitted by objects and converts it into a digital temperature reading using a thermopile detector and internal ADC. It can measure both: **Ambient temperature** and **Object temperature**

Figure 7: MLX90614 Infrared Temperature Sensor

Pin Configuration

Pin	Description
VCC	Power supply (3.3V or 5V)
GND	Ground
SDA	I^2C data line
SCL	I^2C clock line

```
#include <Wire.h>
#include <Adafruit_MLX90614.h>
// Create an object for the sensor
Adafruit_MLX90614 mlx = Adafruit_MLX90614();
void setup() {
Serial.begin(115200);
Serial.println("MLX90614 Infrared Temperature Sensor Test");
if (!mlx.begin()) {
Serial.println("Error connecting to MLX90614. Check wiring!");
while (1);
}
}
void loop() {
// Read temperatures
double ambientTemp = mlx.readAmbientTempC();  // Ambient
temperature in °C
double objectTemp = mlx.readObjectTempC();    // Object
temperature in °C
Serial.print("Ambient Temp: ");
Serial.print(ambientTemp);
Serial.println(" °C");
Serial.print("Object Temp:  ");
Serial.print(objectTemp);
Serial.println(" °C");
Serial.println("----------------------");
delay(1000);
}
```

Figure 8: MLX90614 Infrared Temperature Sensor Interfacing Code

3 Introduction to IoT Communication Protocols

3.1 Introduction to IoT Communication Protocols

In IoT systems, reliable communication between devices, sensors, and backend servers is essential. Protocols define the rules for data exchange, ensuring efficiency, security, and real-time performance. Selecting the right protocol is critical, especially for resource-constrained devices with limited power, memory, and bandwidth.

During the internship, multiple protocols were explored, including **HTTP, MQTT, CoAP, XMPP, and AMQP**, to evaluate their suitability for various IoT scenarios. Practical implementation provided hands-on experience in device-to-cloud communication, data collection, and visualization, highlighting the importance of choosing efficient and scalable communication methods in IoT systems.

1. HTTP (Hypertext Transfer Protocol) HTTP is a **request-response protocol** widely used for web communication.

- Devices act as clients sending data to servers or cloud platforms using GET or POST requests.
- Suitable for low-frequency or occasional data transmission.
- Example: Sending sensor readings to ThingSpeak or other web dashboards.

Advantages:

- Simple, universally supported

Limitations:

- Higher overhead, less efficient for frequent updates

2. MQTT (Message Queuing Telemetry Transport)
MQTT is a **lightweight publish-subscribe messaging protocol** designed for constrained devices and low-bandwidth networks.

- Sensors (publishers) send data to a broker; subscribers receive updates in real time.
- Ideal for frequent, real-time IoT updates.

Advantages:

- Low overhead, efficient, real-time capable

Limitations:

- Requires a broker, slightly complex setup

3.2 HTTP Protocol with Soil Sensor

The Soil Moisture Sensor was connected to a NodeMCU/Pico W to measure soil moisture levels and send data to a Raspberry Pi via HTTP POST requests to a Flask API. The received values were printed on the Serial Monitor for observation. While reliable, HTTP showed higher sending times due to request overhead, affecting overall speed.

humansensor_http.ino.ino

```
#include <ESP8266WiFi.h>
#include <ESP8266HTTPClient.h>
#include <ArduinoJson.h>

const char* ssid = "Redmi";
const char* password = "Twinkle@1234";

const char* serverUrl = "http://192.168.137.77:1880/human";  // Update this
const int sensorPin = D1;  // PIR sensor connected here

void setup() {
  Serial.begin(115200);
  pinMode(sensorPin, INPUT);

  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
```

Output Serial Monitor

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM5') New Line 115200 baud

```
12:03:21.075 -> ==============
12:03:29.026 -> HTTP Response code: 200
12:03:29.026 -> ==============
12:03:29.026 -> Presence: 1
12:03:29.026 -> Collecting Time: 0 ms
12:03:29.026 -> Processing Time: 1 ms
12:03:29.026 -> Sending Time: 2951 ms
12:03:29.026 -> Total Time: 2952 ms
12:03:29.026 -> ==============
```

3.3 MQTT Protocol with Soil Sensor

The Soil Moisture Sensor data from NodeMCU/Pico W was sent to a Raspberry Pi using MQTT, with the microcontroller as a publisher and a Flask subscriber printing the readings on the Serial Monitor. MQTT achieved lower sending and total times thanks to its lightweight design and persistent connection, making it faster for IoT use.persistent connection, making it faster for IoT use.

```
#include <Wire.h>
#include <Adafruit_MLX90614.h>
#include <ESP8266WiFi.h>
#include <PubSubClient.h>

Adafruit_MLX90614 mlx = Adafruit_MLX90614();

// Wi-Fi credentials
const char* ssid = "Redmi";
const char* password = "Twinkle@1234";

// MQTT broker (Raspberry Pi IP)
const char* mqtt_server = "192.168.137.77";  // Replace with your Pi IP

WiFiClient espClient;
PubSubClient client(espClient);

void setup_wifi() {
  delay(100);
```

Output Serial Monitor

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM5') New Line 115200 baud

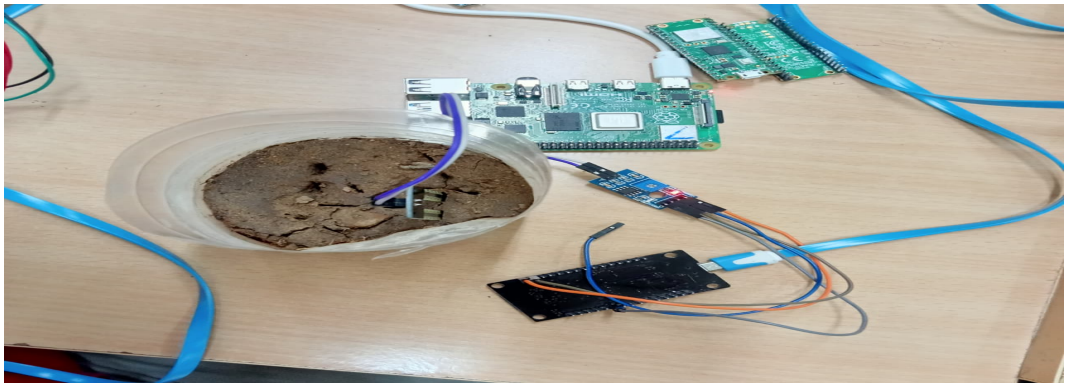
```
10:55:24.480 -> collecting_time: 1
10:55:24.480 -> processing_time: 1
10:55:24.480 -> -------------
10:55:29.428 -> {"sensor":"mlx90614","ambient":1037.55,"object":28.27,"collecting_time_ms":2,"processing_time_ms":0,"timestamp":310978}
10:55:29.428 -> Sending time: 1
10:55:29.428 -> Total time: 4
10:55:29.428 -> collecting_time: 2
10:55:29.428 -> processing_time: 0
10:55:29.428 -> -------------
```

3.4 Experimental Time Analysis

For performance comparison, both HTTP and MQTT protocols were implemented for transmitting data from a Soil Moisture Sensor connected to an IoT device (Raspberry Pi Pico W / NodeMCU) to a Raspberry Pi server.

Metrics Measured :

Data Collection Time – Time taken by the sensor to read and prepare data.
Data Sending Time – Time taken to transmit data to the server.
Data Processing Time – Time taken to store/process data at the server.
Total Time – Sum of all above times.



Result:

Protocol	Collection Time (ms)	Sending Time (ms)	Processing Time (ms)	Total Time (ms)
HTTP	0	2951	1	2952
MQTT	2	1	1	4

Table 1: Comparison of HTTP and MQTT Protocol Performance

Analysis: The experimental results revealed that MQTT significantly reduced both sending and processing times, making it more suitable for real-time IoT systems. Due to its lower total transmission time, MQTT enabled faster and more efficient data updates on dashboards such as Grafana and ThingSpeak. Additionally, the use of Quality of Service (QoS) levels in MQTT ensured reliable data delivery with minimal impact on performance, further enhancing its effectiveness in time-sensitive applications.

4 Advanced MQTT and QoS Level Analysis

To understand the impact of MQTT's Quality of Service (QoS) levels on message reliability and delivery guarantees, I conducted a detailed analysis of each level. MQTT offers three QoS levels, each with distinct trade-offs between performance and assurance.

4.1 QoS 0 (At most once)

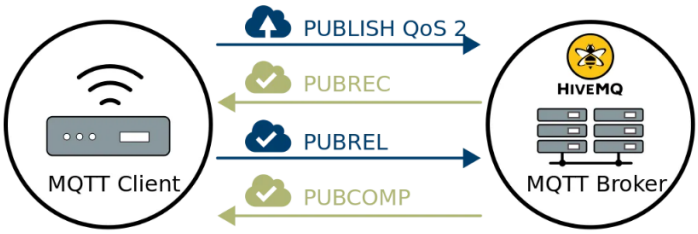
This level provides the fastest message delivery with minimal overhead but does not guarantee that the message will reach the receiver. It is suitable for scenarios where occasional data loss is acceptable, such as non-critical sensor updates.

4.2 QoS 1 (At least once)

QoS 1 ensures that the message is delivered at least once to the receiver. However, it may result in duplicate messages due to retransmissions. This level strikes a balance between reliability and performance, making it suitable for most IoT applications.

4.3 QoS 2 (Exactly once)

QoS 2 guarantees that each message is delivered exactly once, eliminating duplicates entirely. While it provides the highest level of reliability, it also introduces the most overhead and latency. This level is ideal for critical systems where data integrity is paramount.



4.4 Observations

Send a fixed number of messages at QoS 0, 1, and 2.
Count how many messages were sent vs received.
Compare delivery success rate for each QoS.

```
import paho.mqtt.client as mqtt
import time
broker = "localhost"  # or Raspberry Pi IP
topic = "qos_test"
qos_level = 0  # change to 0, 1, or 2
num_messages = 100
client = mqtt.Client()
client.connect(broker, 1883, 60)
for i in range(num_messages):
message = f"Message {i+1} at {time.time()}"
client.publish(topic, message, qos=qos_level)
print(f"Sent: {message}")
time.sleep(0.1)  # short delay to avoid flooding
client.disconnect()
```

```
import paho.mqtt.client as mqtt
broker = "localhost"
topic = "qos_test"
qos_level = 0  # same as publisher
received_count = 0
messages = []
def on_message(client, userdata, msg):
global received_count
received_count += 1
messages.append(msg.payload.decode())
print(f"Received: {msg.payload.decode()}")
client = mqtt.Client()
client.on_message = on_message
client.connect(broker, 1883, 60)
client.subscribe(topic, qos=qos_level)
try:
client.loop_forever()
except KeyboardInterrupt:
print(f"\nTotal Messages Received: {received_count}")
```

Figure 9: code for publisher and subscriber

QoS Level	Messages Sent	Messages Received	Delivery Success Rate	Notes
0	100	94	94%	Some messages lost due to no acknowledgment.
1	100	100	100%	All delivered but duplicates possible in unstable network.
2	100	100	100%	All delivered exactly once, slightly higher delay.

Based on experimental analysis, QoS 0 proved to be highly efficient for transmitting non-critical data due to its minimal overhead. QoS 1 emerged as a practical choice for applications requiring reliable delivery without excessive latency. QoS 2, although the most robust, added noticeable delay and is best reserved for systems where message duplication must be strictly avoided. These insights helped me select appropriate QoS levels based on the criticality and performance requirements of different IoT scenarios.

5 Wireless Communication Using LoRa

5.1 Overview of LoRa (RA-02) module

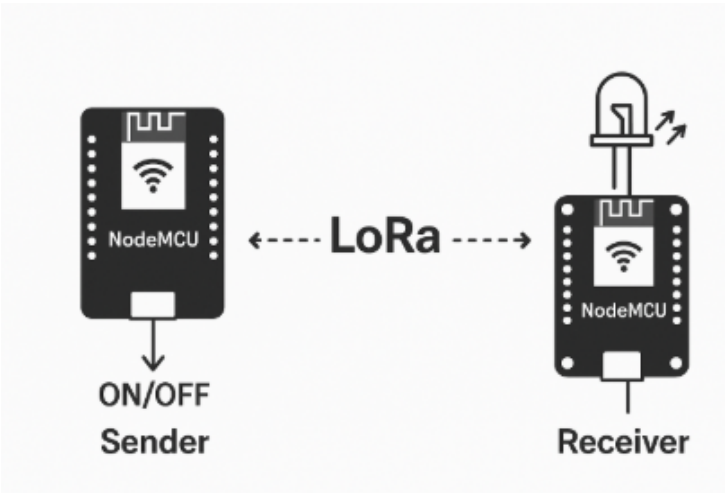
To extend the communication range and reduce dependency on Wi-Fi, I implemented data transfer using LoRa (RA-02) modules. The setup involved configuring a LoRa transmitter and receiver using Raspberry Pi, which enabled sensor data transmission over long distances without relying on internet connectivity. This approach allowed me to explore the potential of LoRa's low-power, long-range capabilities, particularly beneficial for deployments in remote or infrastructure-limited areas.

Key Features

- Supports long range communication (several kilometers in open areas)
- Low power consumption, suitable for battery-powered devices
- Uses spread spectrum modulation for better interference resistance
- Operates generally in 433 MHz / 868 MHz / 915 MHz ISM bands (depending on regional frequency)
- Communicates using SPI interface with microcontrollers

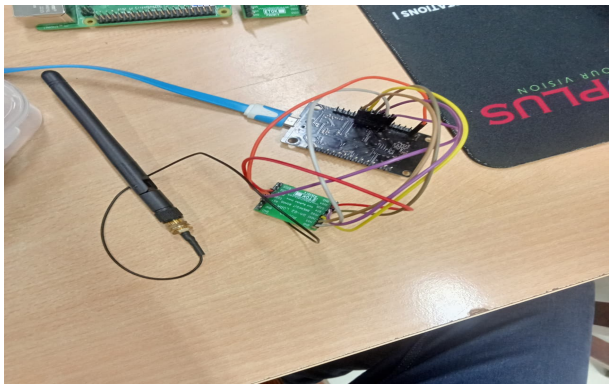
Working Principle

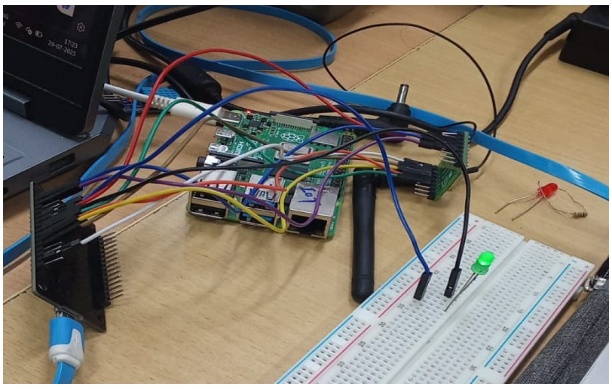
The RA-02 module uses LoRa modulation, which is a form of Chirp Spread Spectrum (CSS). This technique spreads the signal over a wider bandwidth, allowing long-range and robust communication. Devices can send and receive small packets of data over long distances with minimal power consumption.



5.2 LED Blinking via LoRa Transmitter–Receiver Setup

LED blinking via a LoRa transmitter–receiver setup demonstrates wireless long-range communication using LoRa technology. In this setup, two LoRa modules (such as RA-02) are used, one acting as a transmitter and the other as a receiver.
At the transmitter side, a microcontroller sends control data (for example, ON or OFF commands) to the LoRa module. This data is transmitted wirelessly over long distances using LoRa modulation, which provides low power consumption and high resistance to interference.
At the receiver side, the LoRa module receives the transmitted data and forwards it to the connected microcontroller. Based on the received command, the microcontroller controls the LED. If an ON command is received, the LED turns ON; if an OFF command is received, the LED turns OFF or blinks accordingly.
This setup demonstrates how LoRa technology can be used for remote device control, low-power wireless communication, and IoT applications where long-range communication is required without relying on the internet or cellular networks.





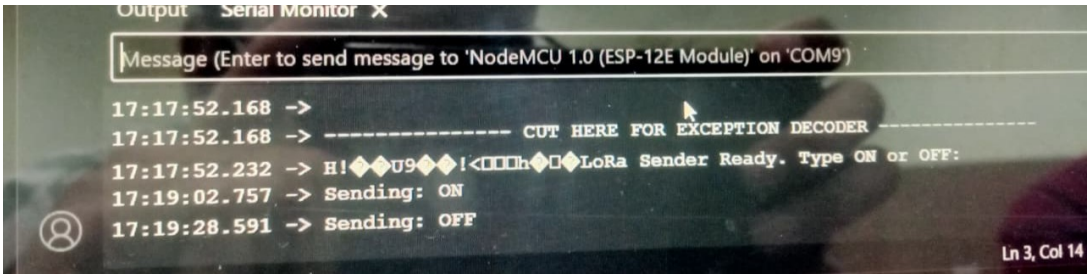


Figure 10: transmitter transmit ON/OFF message to reciver

```
#include <LoRa.h>
#define NSS_PIN 15
#define RST_PIN 16
#define DIO0_PIN 5
void setup() {
Serial.begin(9600);
LoRa.setPins(NSS_PIN, RST_PIN, DIO0_PIN);
if (!LoRa.begin(433E6)) {
Serial.println("LoRa init failed!");
while (1);
}
Serial.println("LoRa Sender ready");
}
void loop() {
LoRa.beginPacket();
LoRa.print("ON");
LoRa.endPacket();
delay(5000);
LoRa.beginPacket();
LoRa.print("OFF");
LoRa.endPacket();
delay(5000);
}
```

```
#include <SPI.h>
#include <LoRa.h>
#define NSS_PIN 15
#define RST_PIN 16
#define DIO0_PIN 5
#define LED_PIN 2 // Onboard LED
void setup() {
Serial.begin(9600);
pinMode(LED_PIN, OUTPUT);
LoRa.setPins(NSS_PIN, RST_PIN, DIO0_PIN);
if (!LoRa.begin(433E6)) {
Serial.println("LoRa init failed!");
while (1);
}
Serial.println("LoRa Receiver ready");
}
void loop() {
int packetSize = LoRa.parsePacket();
if (packetSize) {
String msg = "";
while (LoRa.available()) {
msg += (char)LoRa.read();
}
```

Figure 11: code part for tansmitter and reciever :

Through this implementation, I gained practical insights into LoRa's suitability for wireless IoT communication in domains such as agriculture, remote sensing, and industrial monitoring. The experience demonstrated how LoRa can serve as a robust alternative to traditional wireless protocols, especially in scenarios where energy efficiency and extended range are critical.

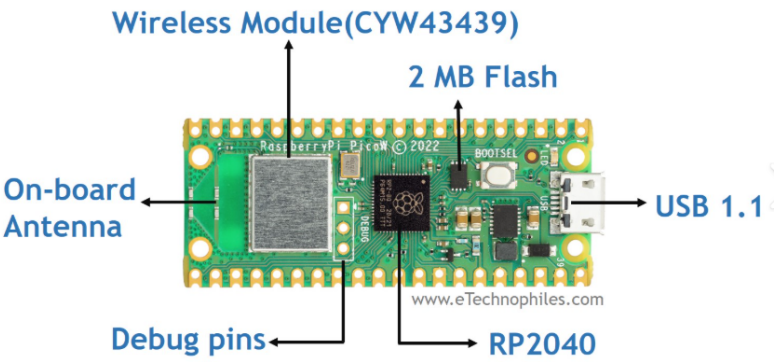
6 IoT-Based Environmental Monitoring System

This project focuses on building a complete **IoT system** for monitoring environmental parameters such as **temperature** and **humidity** using the **DHT11 sensor**. The hardware setup included **NodeMCU** and **Raspberry Pi Pico W** as sensor nodes, while a **Raspberry Pi** served as the central processing and visualization unit. Data management and visualization were carried out using **Node-RED**, **InfluxDB**, **Grafana**, and **ThingSpeak**. Both **HTTP** and **MQTT** protocols were implemented to transmit sensor data, enabling performance comparison.

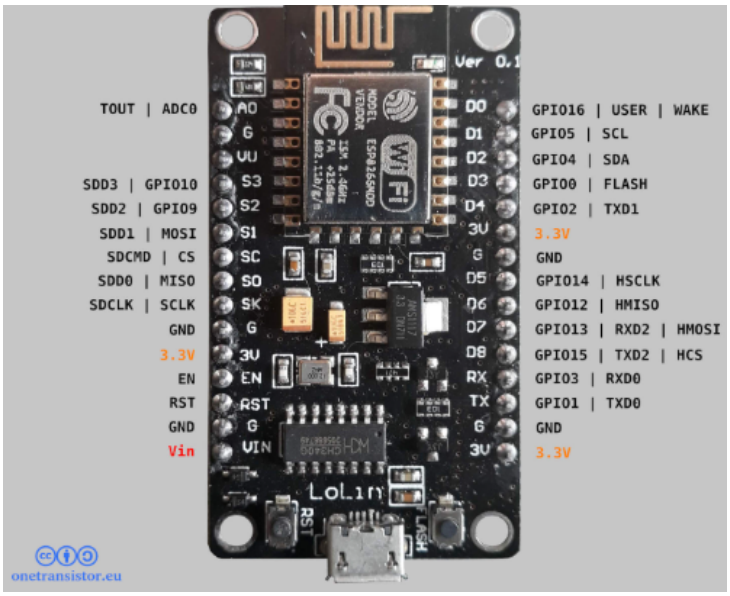
6.1 Hardware used:

Raspberry Pi The Raspberry Pi is a small, affordable single-board computer used for programming, electronics projects, and IoT applications. It runs Linux-based OS like Raspberry Pi OS and can connect sensors, cameras, and other peripherals.

Raspberry Pi Pico W The Pico W is a low-cost microcontroller board from Raspberry Pi with Wi-Fi capability (based on the RP2040 microcontroller). It is ideal for low-power IoT and embedded systems.



NodeMCU (ESP8266) NodeMCU is a low-cost microcontroller board with built-in Wi-Fi, based on the ESP8266 chip. It is popular for IoT applications where you need wireless communication.



6.2 Data Collection and Transmission

The DHT11 sensors connected to NodeMCU and Pico W collected environmental data, which was sent to the Raspberry Pi over **Wi-Fi**. Two communication methods were tested: **HTTP POST** for direct client-server interaction and **MQTT** for lightweight publish/subscribe messaging. This ensured **continuous real-time data logging** and reliable monitoring.

6.3 Node-RED for Data Flow Management

Node-RED was deployed on the Raspberry Pi to manage the flow of data from sensor nodes to storage and visualization systems. It allowed:

- Receiving data via HTTP or MQTT
- Parsing and processing incoming JSON payloads
- Storing readings in InfluxDB

Additional automation included sending alerts when **temperature thresholds** were exceeded. The visual programming interface made integration and modification fast and efficient.

6.4 InfluxDB for Time-Series Storage

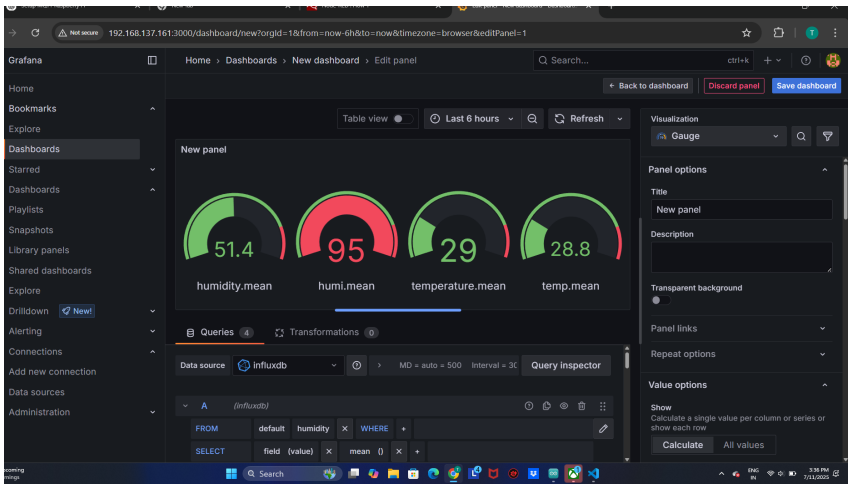
InfluxDB acted as a time-series database, storing readings with timestamps, sensor IDs, and measured values. **Buckets** were created for organizing data, and the concepts of **measurements**, **fields**, and **tags** were applied for structured storage. The connection between Node-RED and InfluxDB enabled automated data insertion.

6.5 Grafana for Real-Time Visualization

Grafana was integrated with InfluxDB to create **interactive dashboards**. These dashboards displayed:

- Line charts for tracking changes over time
- Gauges and bar charts for quick visual indicators

Custom panels and **alert thresholds** were configured to assist in performance monitoring and troubleshooting.

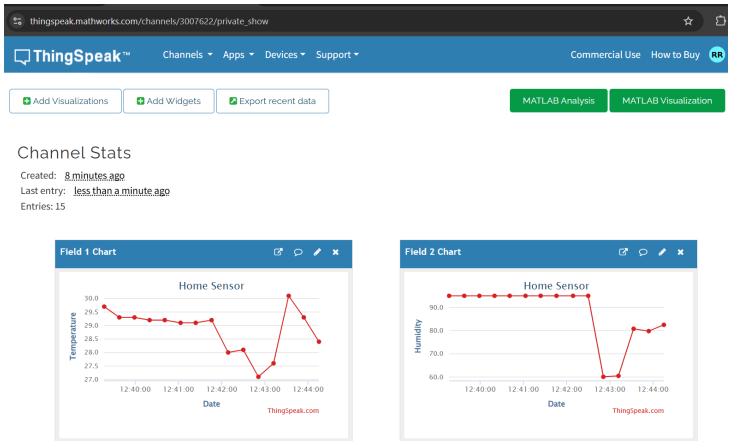


6.6 ThingSpeak for Cloud Monitoring

To extend monitoring capabilities beyond local access, **ThingSpeak** was used for cloud integration. This allowed:

- Remote access to sensor data from any location
- Public sharing of dashboards via URL

Data was transmitted using both **HTTP POST** and **MQTT**, ensuring flexibility and redundancy in cloud storage.



7 Docker-Based IoT Data Processing and Visualization System

7.1 Overview of Docker

Docker is an open-source platform that enables packaging applications and their dependencies into lightweight, portable **containers**. These containers ensure that software runs consistently across different computing environments, eliminating compatibility issues. Unlike traditional virtual machines, containers share the host operating system's kernel, making them **faster** and **more resource-efficient**.
In this project, Docker was used to deploy three essential services inside separate containers: **InfluxDB** (time-series database for storing sensor data), **Flask API** (backend service to receive data from **NodeMCU/Pico W**), and **Grafana** (interactive dashboard for visualization).
These services were defined in a single `docker-compose.yml` file, allowing the entire IoT stack to be launched with a single command.

7.2 Benefits of Modular Docker-Based Deployment

Isolation of Services: Each application ran in its own container, preventing dependency conflicts and improving system stability.
Reproducibility: The same Docker images could be deployed across development, testing, and production environments with identical configurations.
Scalability: Individual services such as the MQTT broker or database could be scaled independently based on system load.
Ease of Maintenance: Containers could be updated, restarted, or replaced without disrupting the entire IoT system.
Rapid Deployment: Docker significantly reduced setup time, enabling faster experimentation and development cycles.

7.3 Services in Docker Containers

Installed **Docker** and **Docker Compose**. Created a project directory with application code, `Dockerfiles`, and a `docker-compose.yml` file.
Built separate containers for **Flask**, **InfluxDB** (with persistent volumes), and **Grafana**. Connected services via **Docker networking**. Launched setup using `docker-compose up -d` and tested:

- Flask → Port **5000**
- InfluxDB → Port **8086**
- Grafana → Port **3000**

7.4 Docker Container Status Output

```
CONTAINER ID  IMAGE              COMMAND           CREATED        STATUS       PORTS          NAMES
a3c7b21f8d4e  grafana/grafana    "/run.sh"       2 minutes ago  Up 2 minutes 0.0.0.0:3000/tcp grafana
b2d4f89e6a1c  my-flask-api-image "python app.py" 3 minutes ago  Up 3 minutes 0.0.0.0:5000/tcp flask_api
c1a2b3c4d5e6  influxdb:1.8       "/entrypoint.sh"4 minutes ago  Up 4 minutes 0.0.0.0:8086/tcp influxdb
```

```
FROM python:3.9
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
CMD ["python", "app.py"]
```

```
docker start influxdb flask_api grafana
```

```
docker-compose up -d
```

```
version: '3'
services:
flask_api:
build: ./api
ports:
- "5000:5000"
depends_on:
- influxdb
influxdb:
image: influxdb:1.8
ports:
- "8086:8086"
volumes:
- influxdb_data:/var/lib/influxdb
grafana:
image: grafana/grafana
ports:
- "3000:3000"
depends_on:
- influxdb
volumes:
influxdb_data:
```

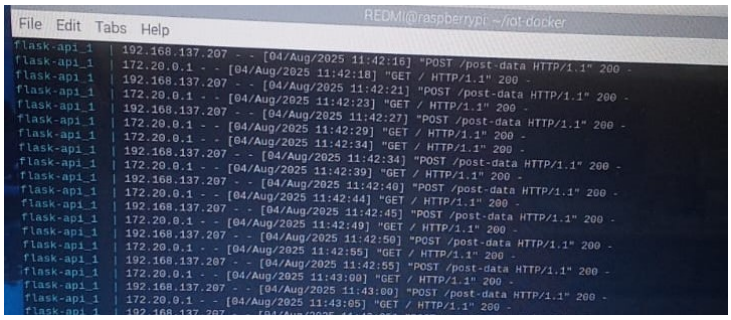
Figure 12: dockerfile (YAML file)

Use of Docker Compose

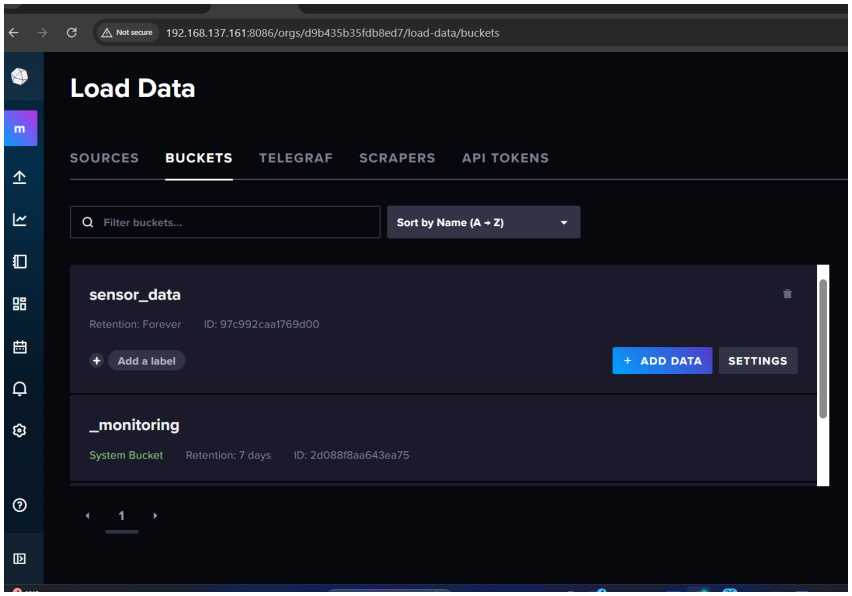
To manage multiple containers efficiently, Docker Compose was used to define and run multi-container applications. It simplified service orchestration by allowing all configurations—networking, volumes, and environment variables—to be defined in a single YAML file. This approach improved readability, maintainability, and deployment automation.

7.5 How Docker Works in This Project

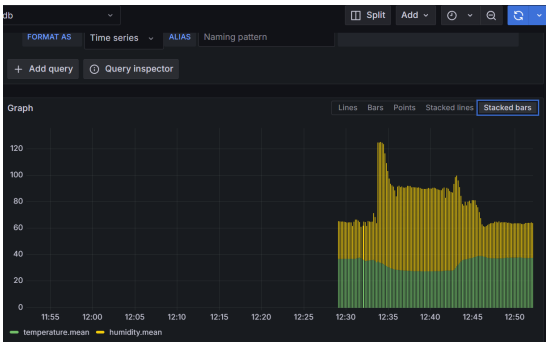
The Docker setup enabled seamless integration between data collection, storage, and visualization:
Sensor Data Collection: The **DHT11** sensor connected to NodeMCU or Pico W measured temperature and humidity values. **Sending Data to Flask API:** Sensor readings were transmitted over HTTP POST to the Flask API container running on Raspberry Pi.

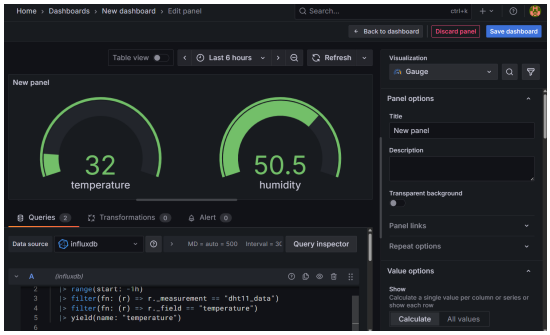


Storing Data in InfluxDB: The Flask API processed incoming readings and inserted them into the InfluxDB container.**InfluxDB** acted as a time-series database, storing readings with timestamps, sensor IDs, and measured values. **Buckets** were created for organizing data, and the concepts of **measurements**, **fields**, and **tags** were applied for structured storage.



Visualizing Data in Grafana: The Grafana container was linked to InfluxDB, allowing real-time temperature and humidity graphs to be displayed on a dashboard. This containerized approach ensured **easy deployment**, **scalability**, and **consistent performance**, regardless of the host system.





8 Conclusion

This internship project focused on the design and implementation of a scalable and modular Internet of Things (IoT) monitoring system. The system architecture integrated microcontrollers including Raspberry Pi 4, NodeMCU (ESP8266), and Raspberry Pi Pico W to support distributed sensor data acquisition and wireless communication.

Various environmental sensors were interfaced to collect data, such as DHT11 for temperature and humidity, PIR for motion detection, ultrasonic sensors for distance measurement, and infrared sensors for non-contact temperature readings. Data transmission was carried out using both HTTP and MQTT protocols, enabling a performance comparison between client-server and publish-subscribe communication models. Detailed analysis of MQTT Quality of Service (QoS) levels was conducted to understand delivery guarantees and network efficiency.

On the software side, Node-RED was used to manage data flows and trigger actions. InfluxDB served as the time series database for efficient storage, while Grafana was utilized for real-time visualization and dashboard creation. Cloud-based integration was also implemented using ThingSpeak to extend monitoring capabilities beyond local networks.

To ensure modular deployment and reproducibility, core services, including back-end API, database, and visualization tools, were containerized using Docker. This approach enabled simplified system orchestration and scalability. Additionally, long-range wireless communication was implemented using LoRa (RA-02) modules, making the system suitable for remote or infrastructure-constrained environments.

The overall system demonstrated end-to-end IoT functionality, from sensor data collection and transmission to processing, storage, and visualization, highlighting the feasibility of building portable, efficient, and production-ready IoT monitoring solutions.